

Ajuda per la Pràctica 3 Part 2

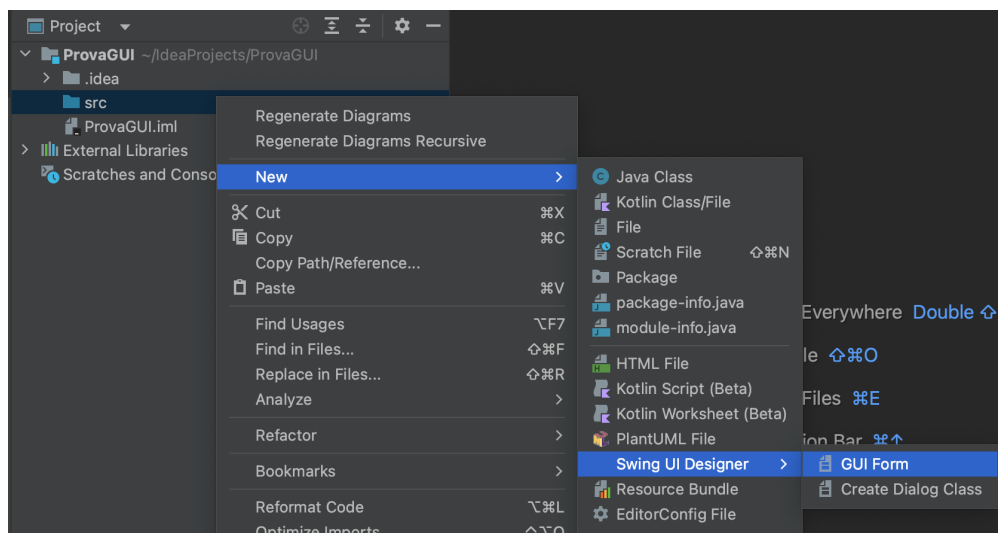
En aquest document s'explica com construir una Interfície Gràfica d'Usuari (GUI) fent servir l'Entorn de Desenvolupament Integrat (IDE) IntelliJ IDEA. Trobareu també ajuda per utilitzar la classe JDialog i JFileChooser.

1. Projecte GUI

1.1. Creació del projecte

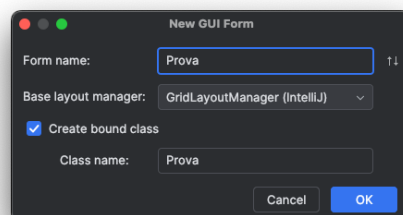
Seguiu els següents passos per tal de crear un projecte GUI amb l'entorn IntelliJ IDEA:

1. Creeu un nou projecte de **Java** "ProvaGUI" sense classe Main (desactivant l'opció "Add sample code").
2. Feu clic dret a la carpeta src -> New -> Swing UI Designer -> GUI Form, tal com es mostra en la imatge:



Si no trobes el "Swing UI Designer" al teu menú, és possible que no estigui instal·lat. Per instal·lar-ho, pots anar a "Settings", després a "Plugins", buscar "Swing UI Designer" i després instal·lar-ho.

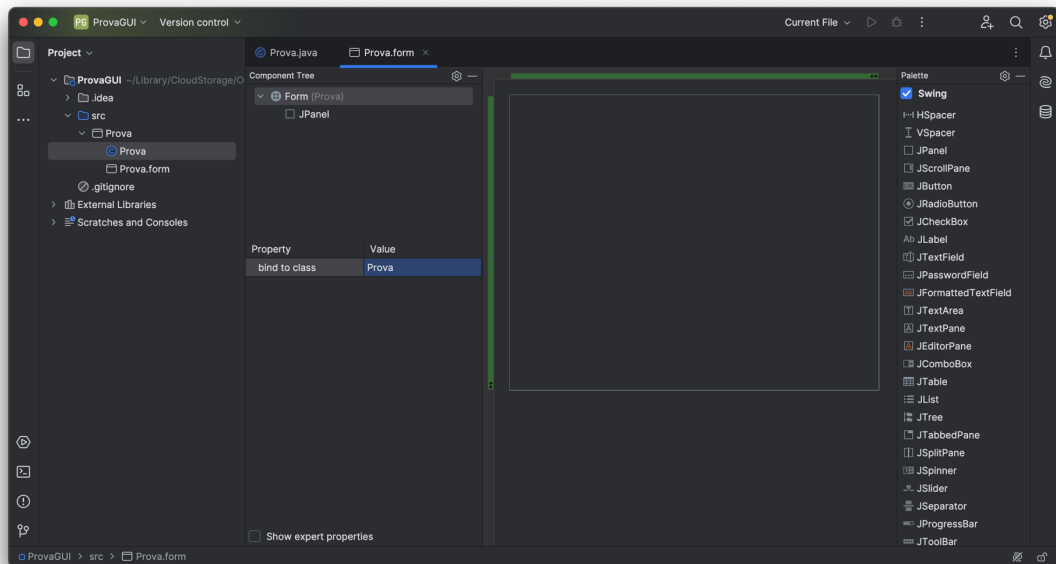
Poseu-li el nom "Prova" a la nova GUI:



Programació 2.

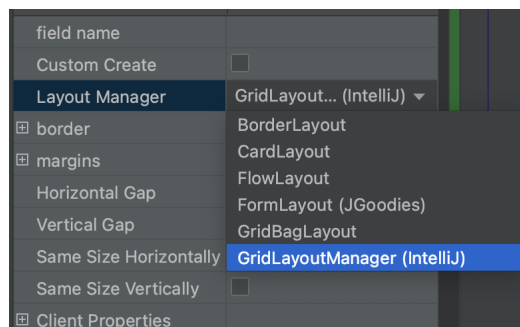
Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB

3. En cas de que no estigui seleccionada per defecte, escolliu l'opció **GridLayoutManager**.
4. IntelliJ crearà un fitxer .form i un fitxer .java que podreu veure en dues pestanyes:
 - La pestanya form visualitza l'aplicació gràfica generada (buida en aquest moment) i conté una paleta (Palette) amb una sèrie d'eines gràfiques per modificar-la.
 - La pestanya java conté el codi base de la classe.



5. En el pas 3 s'indica que escolliu **GridLayoutManager**, però si aneu a la pestanya Prova.form i seleccioneu el panell principal (**JPanel**), a la part inferior esquerra apareixen les propietats d'aquest objecte. Entre altres coses, podem veure que disposem de diverses opcions del Layout Manager i podem modificar la tria. Un **Layout Manager** és el sistema que s'encarrega de col·locar i organitzar automàticament els components (com botons, etiquetes, caixes de text...) dins d'un contenidor (com un JPanel) en una interfície gràfica. Cada tipus de layout segueix unes regles diferents per distribuir els elements. A continuació podeu veure les més comunes:

Nom	Descripció
GridLayoutManager	(Predeterminat) Flexible i fàcil de fer taules, formularis, etc.
BorderLayout	Divideix en North, South, East, West, Center.
FlowLayout	Col·loca components en línia, com paraules en un text.
GridLayout	Graella amb cel·les del mateix tamany.



Programació 2.

Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB

6. Comproveu que el codi de la pestanya Prova.java està buit. Ara aneu a la pestanya Prova.form. Fixeu-vos que el **JPanel** per defecte no té nom (el camp `fieldName` està buit), poseu-li *panelProva*. Fixeu-vos que ara us apareix aquest panell com a atribut de la classe Prova (al codi del fitxer **Prova.java**). Feu **que la classe Prova hereti de JFrame** i afegiu el següent mètode `main` al codi.

```
public class Prova extends JFrame{
    ...

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            Prova prova = new Prova();
            prova.setVisible(true);

        });
    }
    ...
}
```

7. Si executeu ara, s'obrirà una finestra sense cap component a la part superior de la pantalla. Per fixar la posició, la mida i el títol podeu afegir les línies de codi següents al constructor:

```
public Prova(){
    setTitle("Prova GUI");
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setContentPane(panelProva);
    setSize(500,400);
    setLocationRelativeTo(null);
}
```

Torneu a executar per comprovar com es visualitzen les modificacions introduïdes al codi.

1.2. Afegir components gràfics

Com heu pogut veure, quan treballem amb interfícies gràfiques (GUI) al IntelliJ IDEA Ultimate, disposem de dues vistes:

- Vista del codi font (pestanya **.java**)
- Vista de disseny (dissenyador gràfic de l'IDE, pestanya **.form**).

A la vista de disseny disposem de l'eina anomenada Palette. Aquesta paleta conté els diferents components gràfics que podem utilitzar a la nostra aplicació: botons, camps de text, etiquetes, llistes desplegable, arbres, caselles de selecció, etc., i que podem afegir a la nostra GUI arrossegant-les al formulari.

Per començar a construir una interfície senzilla:

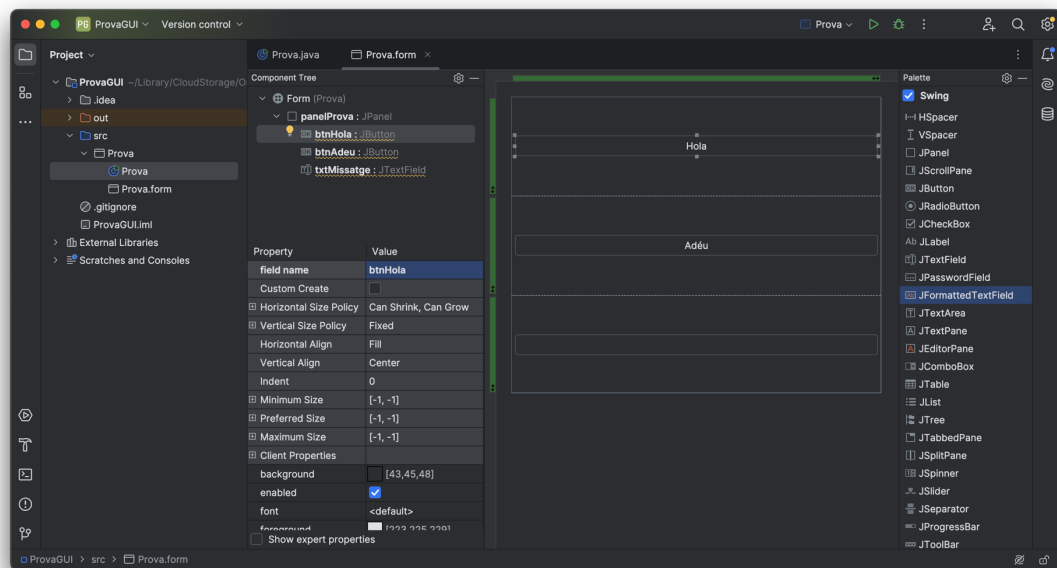
8. Aneu a la pestanya de la vista de Disseny (fitxer Prova.form).
 9. Arrossegueu els següents components des de la paleta fins al formulari:
 1. Un **JTextField** (camp de text)
 2. Dos **JButton** (botons)
 10. Aquesta acció us crearà automàticament al teu codi:
 - Un objecte de tipus **JTextField**
 - Dos objectes de tipus **JButton**
- Obriu el codi font per comprovar-ho.

Programació 2.

Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB

11. Seleccionant cada una de les components afegides al formulari en podem canviar les propietats. Canvieu el text dels botons, modificant "Button" per "Hola" i "Adéu", i en cas de que no estigui buit per defecte elimina el text inicial del **TextField**. Obriu el codi font i fixeu-vos que han aparegut aquestes components a la classe.

En cas de que vulgueu canviar els noms de les variables, ho podeu fer des de la pestanya **.form** modificant el valor del camp "field name" de la llista de propietats de la component, com hem fet abans amb el **JPanel**. Si canvieu els noms des del **.java** es deixaran de correspondre amb els elements de la paleta. Renombrreu les variables de `button1` i `button2` a `btnHola` i `btnAdeu`, i de `textField1` a `txtMissatge`.



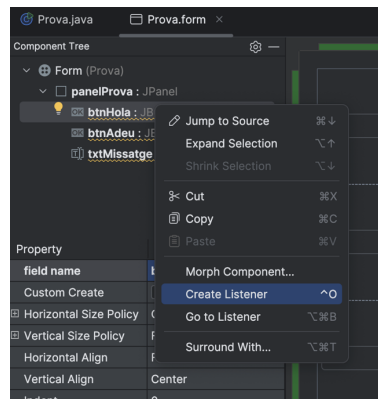
1.3. Afegir escoltadors

Per afegir funcionalitat als botons i altres components de la interfície, hem de vincular-hi escoltadors d'esdeveniments a aquestes components. Això permet que el programa reaccioni quan l'usuari interactua amb els components (com fer clic en un botó o marcar un checkbox).

12. Seleccioneu el botó "Hola" a la vostra vista de disseny.
 - Accediu al menú (botó dret del ratolí sobre el botó) i selecciona **CreateListener > ActionListener**.
 - Això farà que l'IDE generi automàticament un **ActionListener** per al botó "Hola".

Programació 2.

Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB



Un cop seleccionat l'opció, l'IDE obrirà el fitxer de codi font (Prova.java) i us portarà automàticament a la ubicació on has de completar l'escoltador de l'esdeveniment. El mètode `actionPerformed` estarà buit, i aquí és on heu d'afegir el codi que s'executarà quan el botó sigui premut.

```
public class Prova extends JFrame{
    private JPanel panelProva; 2 usages
    private JButton btnHola; 2 usages
    private JButton btnAdéu; 1 usage
    private JTextField txtMissatge; 1 usage

    public Prova(){ 1 usage
        setTitle("Prova GUI");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setContentPane(panelProva);
        setSize( width: 500, height: 400);
        setLocationRelativeTo(null);
        btnHola.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // Mètode a completar
            }
        });
    }
}
```

13. Afegiu a dins del mètode `actionPerformed()` el codi següent per tal de quan es premi el botó el text "Hola" apareixi al camp de text:

```
txtMissatge.setText(btnHola.getText());
```

14. Repetiu els mateixos passos que per al botó "Hola", però ara per al botó "Adéu".

1.4. Afegir JCheckBox i el seu escoltador

15. Ara afegiu un checkbox per activar o desactivar els botons "Hola" i "Adéu". Això permetrà que l'usuari controli si pot interactuar amb els botons o no.

16. Modifiqueu el text del checkbox i nom de la variable:

- Canvia el text de `JCheckBox1` a "Botons Actius".
Canvia el nom de la variable associada al checkbox per `chkBotonsActius`.

17. Configureu l'estat inicial del checkbox i el camp de text:

- Al constructor de la classe `Prova`, afegiu les següents línies per garantir que el checkbox estigui marcat i el camp de text també comenci activat per defecte:
`chkBotonsActius.setSelected(true);`

Programació 2.

Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB

```
txtMissatge.setEnabled(true);
```

18. Afegiu l'escoltador per al checkbox. En concret, afegirem un **ChangeListener** al checkbox per detectar quan es canvia el seu estat (marcat/desmarcat). Així activarem o desactivarem els botons segons l'estat del checkbox. Per fer-ho:

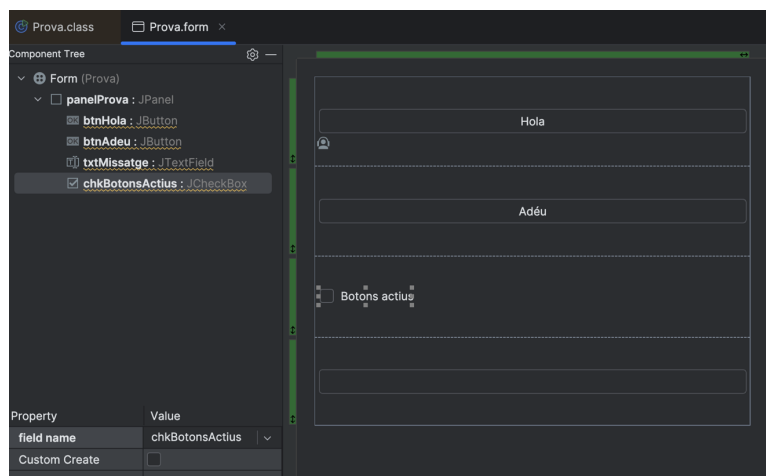
- Des del Prova.form, feu clic dret sobre el checkbox i selecciona **Create Listener > ChangeListener** per generar el mètode automàticament. Completa el codi del mètode `stateChanged` de la següent manera:

```
chkBotonsActius.addChangeListener(new ChangeListener() {  
    @Override  
    public void stateChanged(ChangeEvent e) {  
        btnHola.setEnabled(chkBotonsActius.isSelected());  
        btnAdeu.setEnabled(chkBotonsActius.isSelected());  
    }  
});
```

Així, quan el checkbox està seleccionat, els botons estaran actius, i quan el checkbox està desmarcat, els botons estaran inactius (desactivats).

19. Després de seguir aquests passos, hauríeu de veure els següents elements a la interfície:

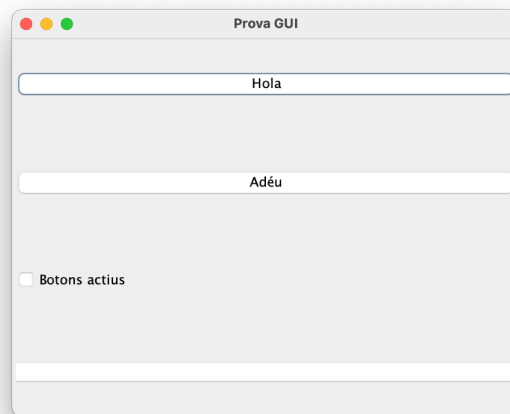
- Un **JTextField** per mostrar el text.
- Dos **JButtons** anomenats "Hola" i "Adéu". Quan es prem un botó, el seu nom apareixerà al **JTextField**.
- Un **JCheckBox** que, quan està seleccionat, permet interactuar amb els botons, i quan està desmarcat, els desactiva.



I per tant, l'aplicació es veurà així:

Programació 2.

Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB



1.5. Afegir una llista i botons per manipular-la

Ara aprendrem com podem mostrar elements d'una llista i eliminar elements de la mateixa llista. Per fer-ho seguim els següents passos:

20. Creeu un botó "Mostrar Llista". Canvieu el text del botó a "**Mostrar Llista**" i el nom de la variable associada al botó a **btnMostrar**.

21. Afegiu un component de tipus **JList**:

- Des de la paleta d'eines, arrossegueu un control de tipus **JList** i afegiu-lo a la finestra.
- Canvieu el nom de la variable associada a aquest control de llista a `lstLlista1`.

22. Afegiu un atribut a la classe Prova:

- A la classe **Prova**, afegiu l'atribut següent per emmagatzemar la llista (amb els imports que us siguin necessaris):
`ArrayList<A> llista;`
- Afegiu la classe **A**, que tindrà dos atributs `x` i `y`, i un mètode `toString()` per mostrar el contingut d'aquesta classe de manera llegible:

```
class A {
    int x;
    int y;

    public A(int x, int y){
        this.x=x;
        this.y=y;
    }
    @Override
    public String toString(){
        return "x= " + x + "\ny= " + y;
    }
}
```

- Pots inicialitzar la llista dins del constructor fent:
`llista = null;`

Programació 2.

Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB

23. Afegiu el mètode `afegirDadesTest()` a la classe **Prova**:

- El mètode `afegirDadesTest()` per inicialitzar la llista amb alguns elements de prova:

```
private void afegirDadesTest() {  
    llista = new ArrayList<A>();  
    llista.add(new A(1,2));  
    llista.add(new A(3,4));  
    llista.add(new A(5,6));  
}
```

- Crida aquest mètode al constructor de la classe **Prova** per inicialitzar les dades de prova.

24. Mostreu la llista quan es premi el botó "Mostrar Llista":

- Afegiu un escoltador d'esdeveniments al botó "**Mostrar Llista**". Implementa el mètode `omplirLlista()` per omplir el component **JList** amb les dades de l'**ArrayList**, i fes que es cridi al clicar el botó.

```
void omplirLlista() {  
    DefaultListModel model = new DefaultListModel();  
    model.clear();  
    for(A item: llista) {  
        model.addElement(item);  
    }  
    lstLlista1.setModel(model);  
}
```

25. Afegiu un botó per eliminar elements de la llista

- Creeu un botó: objecte de tipus **JButton**
- Canvieu el text que conté el botó a "**Eliminar**"
- Canvieu el nom de la variable a `btnEliminar`

26. Feu que el botó **Eliminar** estigui desactivat per defecte:

- Al constructor de la classe **Prova**, afegiu la següent línia per assegurar-te que el botó d'eliminar està desactivat quan no s'ha seleccionat cap element de la llista:
`btnEliminar.setEnabled(false);`

27. Activeu el botó d'eliminar quan es selecciona un element de la llista. Per fer-ho s'ha d'afegir un escoltador a la llista `lstLlista1`:

- Des de **Prova.form**, feu clic dret sobre la llista i selecciona **Create Listener > ListSelectionListener** per generar el codi automàticament.
- Implementeu el codi del mètode `valueChanged` per activar el botó **Eliminar** només quan un element de la llista estigui seleccionat afegint a l'escoltador:

```
public void valueChanged(ListSelectionEvent e) {  
    if (!e.getValueIsAdjusting()) {  
        btnEliminar.setEnabled(!lstLlista1.isSelectionEmpty());  
    }  
}
```

28. Afegiu la funcionalitat per eliminar l'element seleccionat de la llista. Per fer-ho s'ha d'afegir un escoltador al botó `btnEliminar`:

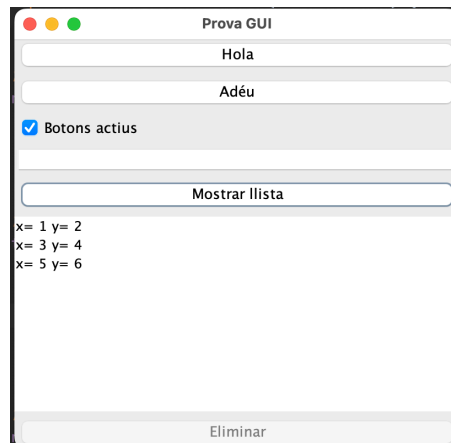
- Des de **Prova.form**, afegiu l'escoltador d'esdeveniments (**ActionEvent**) al botó **Eliminar** i implementa el mètode `actionPerformed()` de la següent manera:

Programació 2.

Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB

```
public void actionPerformed(ActionEvent e) {
    for (Object item : lstLlista1.getSelectedValuesList()) {
        A elementSeleccionat = (A) item;
        llista.remove(elementSeleccionat);
    }
    omplirLlista();
}
```

Ara l'aplicació es veu de la següent manera:



2. JDialog

Les finestres de diàleg són subfinestres independents de la finestra principal de l'aplicació, però gestionades per aquesta. A continuació crearem una aplicació que omple uns camps de text agafant les dades d'un **JDialog**.

1. Comencem creant un nou paquet GUI Form anomenat **DialogDemo**, com hem fet abans (src -> New -> Swing UI Designer -> GUI Form). Seguiu els mateixos passos que amb la creació de la Prova anterior per tal de que us aparegui una pantalla buida al executar el codi. Es a dir, heu d'incloure un mètode main al codi DialogDemo.java i heu de donar un valor al "field name" del JPanel que teniu creat al DialogDemo.form.

```
public class DialogDemo extends JFrame{
    ...

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            DialogDemo dialogDemo = new DialogDemo();
            dialogDemo.setVisible(true);
        });
    }
    ...
}
```

Programació 2.

Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB

2. Creeu la classe **Arxiu** dins la classe **DialogDemo**, i feu que el constructor de **DialogDemo** creï una instància d'un arxiu. A continuació us proporcionem la classe **Arxiu**:

```
public class Arxiu {
    private String nom;
    private String tipus;

    public String getNom() {
        return nom;
    }
    public void setNom(String nom) {
        this.nom = nom;
    }
    public String getTipus() {
        return tipus;
    }
    public void setTipus(String tipus) {
        this.tipus = tipus;
    }
}
```

3. Afegiu dos camps de text **TextField** (**txtNom** i **txtTipus**) i un botó **Button** **btnEditar**. Editeu **txtNom** i **txtTipus** per a que no siguin editables (des del constructor amb el mètode **setEditable(false)**).
4. Afegiu un **JLabel** al costat de cada camp de text per indicar que està relacionat amb Tipus o Nom. Visualment el resultat serà com es mostra a continuació:



5. Ara creeu una **Dialog** class fent: **src -> New -> Swing UI Designer -> Create Dialog class**. Poseu-li de nom **EditarDialog** i desmarqueu les tres opcions de generació automàtica de codi que se us proposen. Anant al **.form** podeu veure que automàticament se us han generat un botó **OK** i un botó **Cancel**.
6. Modifiqueu el constructor per tal que se li passi un **arxiu** per paràmetre i perquè us quedi ben configurat d'aquesta manera:

```
public EditarDialog(JFrame parent, DialogDemo.Arxiu arxiu) {
    super(parent);
    setContentPane(contentPane);
    setSize(600, 500);
    setLocationRelativeTo(parent);
    setModal(true);
}
```

7. Inserteu un camp de text **txtNom** i un **ComboBox** (amb field name "tipus") amb dos elements seleccionables "Video" i "Audio", que es poden afegir amb un doble clic.

Programació 2.

Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB

8. Afegiu escoltadors als botons `buttonOK` i `buttonCancel`. El de `buttonOK` seria el següent:

```
@Override
public void actionPerformed(ActionEvent e) {
    arxiu.setNom(txtNom.getText());
    arxiu.setTipus((String) tipus.getSelectedItem());
    dispose();
}
```

A l'escoltador del `buttonCancel` només caldria fer `dispose()`;

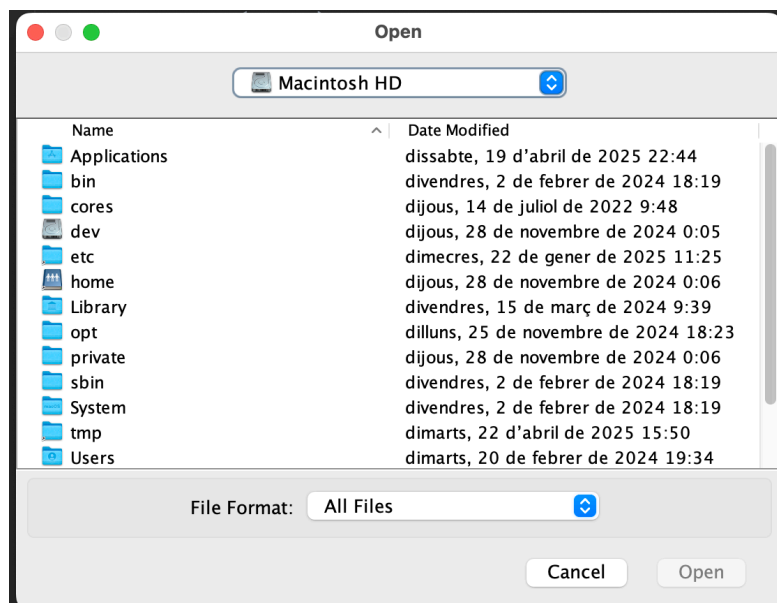
9. Creeu l'event per al botó editar del `JFrame` que s'encarregarà de crear el selector de fitxer i actualitzar els camps de text:

```
btnEdita.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        EditarDialog dialog = new DialogEdito(DialogDemo.this,
arxiu);
        dialog.setVisible(true);
        txtNom.setText(arxiu.getNom());
        txtTipus.setText(arxiu.getTipus());
    }
});
```

Ara hauríeu de poder executar el codi i comprovar el resultat.

3. JFileChooser

Els selectors de fitxers proveeixen una GUI per navegar pel sistema de fitxers. A Java la classe **JFileChooser** ens permet crear una finestra de diàleg, que conté un selector de fitxers, com la que es mostra a continuació:

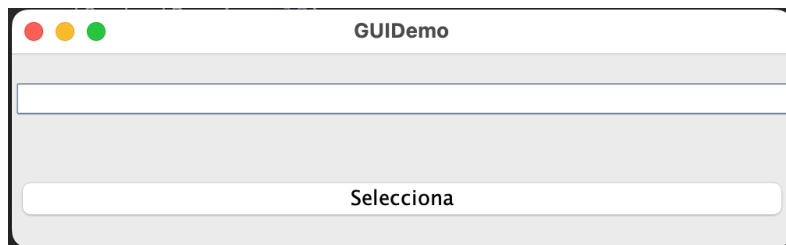


Programació 2.

Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB

A continuació veurem un exemple de selecció de fitxers mitjançant **JFileChooser**, en el que un camp de text s'actualitza en canviar la selecció del fitxer.

1. Creeu un GUI Form que es digui **FileChooser** amb un boto (*btnSelecciona*) i un camp de text (*txtNomFitxer*):



No oblideu que la classe **FileChooser** ha d'estendre de **JFrame**, que s'ha de crear el mètode **main** de la mateixa manera que ho hem fet en els exemples anteriors i que s'han de seguir els passos descrits a la secció 1.1. Creació del projecte.

2. Afegiu l'escoltador del botó selecciona afegint el següent codi a `actionPerformed()`:

```
File fitxer;
//Creació del selector de fitxer
JFileChooser seleccio = new JFileChooser();
//Mostrem la finestra de dialeg
//Resultat emmagazema una constant que indica si s'ha
//seleccionat o no un fitxer
int resultat = seleccio.showOpenDialog(FileChooser.this);
//Assegurem que hi hagi un fitxer seleccionat
if (resultat == JFileChooser.APPROVE_OPTION) {
    //Obtenim el fitxer
    fitxer=seleccio.getSelectedFile();
    //Posem la ruta del fitxer al quadre de text
    txtNomFitxer.setText(fitxer.toString());
}
```

3. Comproveu el funcionament de l'aplicació.

4. Codi generat per IntelliJ

A IntelliJ IDEA, quan utilitzes el **GUI Designer** per crear interfícies gràfiques amb Swing (arrossegant components com JButton, JLabel, etc.), el codi de creació i disposició dels components **no s'escriu manualment per tu** com en una classe .java tradicional.

En lloc d'això, IntelliJ utilitza un fitxer .form (un fitxer XML que representa la disposició gràfica), i genera automàticament el codi Java en temps de compilació. Aquesta generació automàtica es fa a través d'un fitxer auxiliar que **no edites directament**.

En els tres projectes que heu creat en els apartats anteriors heu utilitzat el GUI Designer i heu pogut comprovar com es genera codi automàticament en el .java. Per exemple, haureu vist que per cada component afegida fent servir la paleta (JButton, JTextField, etc.) es crea una variable

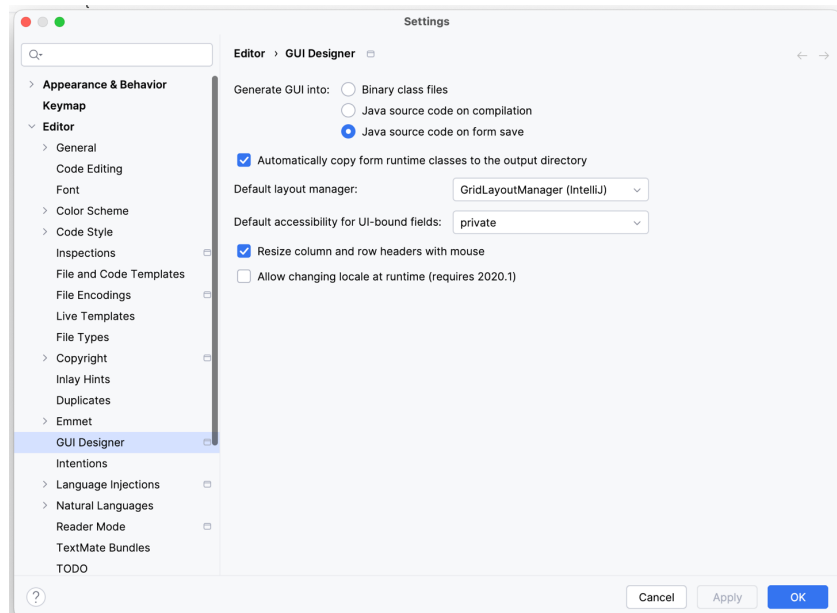
Programació 2.

Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB

de la classe al codi font (.java), però on es crea l'objecte (`new JButton()`)? S'hauria de fer al constructor, veritat?

Tal com està configurat el IntelliJ hi ha part del codi generat que no es mostra en el codi font. Si voleu veure'l podeu fer els següents canvis de configuració:

Aneu a Settings → Gui Designer → “Generated GUI into” → Java source code (en lloc de binary class).



Després, feu “Rebuild Project”.

Tot seguit aneu a Project structure → Modules → Dependencies → + → Library → From Maven. Busqueu “com.intellij”. De la llista trobada, descarqueu “com.intellij:forms_rt:7.0.3”

D'aquesta manera veureu que al final de la classe podeu desplegar el codi generat clicant a “UI Designer generated code” i l'avís informant que és codi generat i que no s'ha d'editar:

```
{
// GUI initializer generated by IntelliJ IDEA GUI Designer
// >>> IMPORTANT!! <<<
// DO NOT EDIT OR ADD ANY CODE HERE!
    $$$setupUI$$$();
}
```

El mètode de suport `$$$setupUI$$$()` s'invoca des del constructor de la classe i s'encarrega d'inicialitzar les components de la GUI.

5. Referències

- [1] <https://www.jetbrains.com/help/idea/design-gui-using-swing.html>
- [2] <https://www.youtube.com/watch?v=aIdIXsi1qTU>

Programació 2.

Grau d'Enginyeria Informàtica. Facultat de Matemàtiques. UB
